

Universidad Gerardo Barrios
Facultad de Ciencia y Tecnología



Informe Final de Proyecto Trabajo de Graduación
Modalidad Preespecialización
en la Especialidad de:

Desarrollo de Aplicaciones con Inteligencia Artificial

ReceiptIA Presentado por:
David Arnoldo Hernández Gómez

Jonathan Elías Gámez Larin

Wilber José Jiménez Ramírez

San Miguel, 23 de 08 de 2026

Índice

Contenido

Índice	2
3. Resumen del proyecto	3
4. Marco conceptual.....	4
4.1 Inteligencia artificial	4
4.2 Inteligencia artificial generativa.....	4
4.3 Modelos de lenguaje de gran escala	5
4.4 Procesamiento de lenguaje natural	5
4.5 Reconocimiento óptico de caracteres (OCR).....	6
4.6 Procesamiento digital de imágenes	6
4.7 Procesamiento inteligente de documentos.....	7
4.8 Extracción y estructuración de información	7
4.9 Detección de anomalías	7
4.10 Automatización de procesos administrativos	8
4.11 APIs REST.....	8
4.12 FastAPI.....	9
4.13 Validación de datos y Pydantic.....	9
4.14 Bases de datos y Firebase Firestore.....	9
4.15 Autenticación y autorización	10
4.16 Docker y contenedorización.....	10
4.17 Integración continua	10
4.18 Observabilidad y logging	11
4.19 Identificación de solicitudes mediante request_id	11
4.20 Monitoreo y Health Checks	11
4.21 Rendimiento y benchmarking	12
4.22 Seguridad de aplicaciones	12
4.23 Limitaciones y riesgos de la inteligencia artificial	13
4.24 Relación de los conceptos con ReceiptIA	13
5. Análisis del problema	14
6. Propuesta de solución.....	15
6.1 Funcionamiento general de la solución.....	15
6.2 Componentes principales	16
6.3 Arquitectura y diseño de la solución	17

6.4 Decisiones técnicas.....	18
6.5 Justificación de la solución.....	19
7. Metodología de desarrollo del proyecto o investigación.....	20
7.1 Análisis y definición de requerimientos.....	20
7.2 Diseño de la arquitectura.....	21
7.3 Desarrollo del backend.....	21
7.4 Implementación del reconocimiento óptico de caracteres.....	22
7.5 Integración de inteligencia artificial.....	22
7.6 Implementación de autenticación y almacenamiento.....	23
7.7 Implementación de pruebas automatizadas.....	24
7.8 Integración continua mediante GitHub Actions.....	25
7.9 Contenedorización mediante Docker.....	26
7.10 Implementación de observabilidad.....	28
7.11 Implementación del monitoreo.....	29
7.12 Evaluación final.....	30
7.12.1 Evaluación de rendimiento.....	30
7.12.2 Evaluación del componente de inteligencia artificial.....	31
7.12.3 Análisis de la evaluación.....	32
8. Resultados alcanzados.....	32
8.1 Funcionamiento general de ReceiptIA.....	32
8.2 Resultado del procesamiento de facturas.....	33
8.3 Resultado de autenticación y almacenamiento.....	34
8.4 Resultado de las pruebas automatizadas.....	34
8.5 Resultado de la integración continua.....	35
8.6 Resultado de la evaluación del componente de inteligencia artificial.....	36
8.7 Resultado del benchmark de rendimiento.....	37
8.8 Resultado de la observabilidad.....	37
8.9 Resultado del monitoreo automatizado.....	39
8.10 Resultado de la contenedorización.....	40
8.11 Síntesis de resultados.....	41
9. Conclusiones generales.....	42
9.1 Conclusión general.....	42
9.2 Resultados concretos.....	42
9.3 Aportes del proyecto.....	42
9.4 Conocimientos adquiridos.....	43

10. Referencias estudiadas	44
----------------------------------	----

3. Resumen del proyecto

ReceiptIA es una aplicación web diseñada para optimizar el análisis y la organización de facturas a través de la tecnología de reconocimiento óptico de caracteres (OCR) y algoritmos de inteligencia artificial.

El objetivo del proyecto es minimizar el tiempo y los errores vinculados al manejo manual de documentos, simplificando la extracción de datos y la detección de inconsistencias potenciales.

El sistema permite la carga de imágenes de facturas o recibos y utiliza Tesseract OCR para obtener el texto, que luego es procesado por Gemini 2.5 Flash para generar información estructurada que incluye datos como comercio, fecha, NIT, subtotal, IVA y total.

Además, cuenta con mecanismos para identificar anomalías y facilitar el análisis de los documentos. La aplicación se compone de un interfaz web y un servidor backend creado con FastAPI, complementado por Firebase Authentication y Firestore. Entre sus funcionalidades destacadas se encuentran el panel de control, el registro de facturas, la organización mediante carpetas y la capacidad de exportar datos a Excel.

Durante su fase de desarrollo, se implementaron técnicas de ingeniería de software tales como validación a través de Pydantic, pruebas automatizadas con Pytest, análisis estático mediante Ruff, integración continua utilizando GitHub Actions y contenedorización con Docker.

En su versión final, también se incorporaron métodos de observabilidad y monitoreo, incluyendo request_id, registro de solicitudes y verificaciones periódicas del endpoint /health. La solución fue sometida a pruebas de rendimiento y de inteligencia artificial. En un benchmark de 20 peticiones, se registraron 16 respuestas exitosas y 4 fallos, resultando en una tasa de error del 20 %, un P50 de 10.68 segundos y un P95 de 13.71 segundos. Además, los cuatro casos utilizados para testar la inteligencia artificial lograron un 100 % de precisión en los campos analizados.

En resumen, ReceiptIA ilustra la implementación efectiva de inteligencia artificial para automatizar el manejo documental y mejorar la gestión de facturas.

Este proyecto permitió al equipo adquirir experiencia en OCR, inteligencia artificial, desarrollo de APIs, gestión de bases de datos, realización de pruebas, Docker, observabilidad, monitoreo y evaluación de modelos.

4. Marco conceptual

4.1 Inteligencia artificial

La inteligencia artificial (IA) representa un ámbito de la informática enfocado en la creación de sistemas que son capaces de ejecutar tareas que tradicionalmente demandan habilidades asociadas a la inteligencia humana.

Entre estas habilidades se incluyen la detección de patrones, la comprensión de información, la elaboración de pronósticos, la organización de datos y el apoyo en la toma de decisiones. El Instituto Nacional de Estándares y Tecnología (NIST) describe la inteligencia artificial como un sistema mecánico que, a partir de objetivos fijados por humanos, tiene la capacidad de realizar pronósticos, sugerencias o decisiones que impactan tanto en entornos físicos como virtuales.

El desarrollo de la inteligencia artificial ha facilitado su integración en diversas áreas, tales como la salud, la educación, la industria, el comercio, las finanzas y la gestión. Su relevancia se relaciona, en gran medida, con la aptitud para procesar volúmenes significativos de información y automatizar tareas que tienden a ser repetitivas o que demandan un análisis continuo.

En el sector empresarial, la inteligencia artificial puede ser empleada como una herramienta para mejorar procesos, disminuir las labores manuales y extraer información de datos que anteriormente requerían un análisis humano. No obstante, su adopción también implica examinar factores relacionados con la exactitud, la seguridad, la privacidad, la transparencia y el manejo de riesgos.

En ReceiptIA, la inteligencia artificial se erige como uno de los elementos fundamentales de la solución. El sistema aplica IA para analizar la información extraída de las facturas y transformarla en datos estructurados que pueden ser posteriormente almacenados, evaluados y estudiados.

4.2 Inteligencia artificial generativa

La inteligencia artificial generativa se refiere a un tipo de sistemas que pueden crear contenido nuevo basándose en los patrones que aprendieron mientras se entrenaban. Estos sistemas son capaces de generar texto, imágenes, sonidos, código y otro tipo de información, según las características del modelo y lo que pueda hacer.

Algo importante de la IA generativa es que no solo clasifica lo que recibe en categorías que ya existen. También puede entender instrucciones, conectar diferentes datos y generar respuestas organizadas según el contexto que se le da.

Usar inteligencia artificial generativa trae consigo ciertos riesgos. El NIST señala que hay peligros en los sistemas de IA generativa que hay que considerar en todo su ciclo: desde que se diseña, se desarrolla, se evalúa y se usa. Entre estos riesgos están la confiabilidad, la seguridad, la privacidad, la transparencia y cómo manejar los posibles errores.

En ReceiptIA, usamos inteligencia artificial generativa para revisar el texto que saca el OCR. En vez de solo guardar todo el texto de una factura, el sistema le pide al modelo que encuentre la información importante y la ponga en un formato que luego el backend pueda usar.

4.3 Modelos de lenguaje de gran escala

Los modelos de lenguaje grandes, o LLM por sus siglas en inglés, son sistemas de inteligencia artificial que sirven para entender y generar texto. Pueden revisar un montón de información escrita y encontrar relaciones entre las diferentes partes de lo que se les da.

Estos LLM se usan para varias cosas, como clasificar cosas, resumir textos, sacar datos, crear contenido, traducir y seguir instrucciones concretas. Cómo de bien funcionen depende de qué tipo de modelo se use, de la información que se les dé en la petición y de cómo esté redactada la instrucción. Una aplicación muy útil de estos modelos es para buscar información.

Un mismo documento puede tener datos de muchas formas distintas, y un modelo de lenguaje puede ayudar a encontrar los puntos importantes y pasarlos a un formato más uniforme.

En ReceiptIA usamos esta idea para procesar facturas que, como sabes, pueden venir de muchas maneras. Después de sacar el texto con OCR (reconocimiento óptico de caracteres), el modelo mira la información y busca unos campos concretos que el sistema define, como quién vende, la fecha, el importe total y otros detalles clave.

4.4 Procesamiento de lenguaje natural

El Procesamiento del Lenguaje Natural, que llamamos PLN (o NLP en inglés), es una parte de la inteligencia artificial que se dedica a manejar el lenguaje humano usando computadoras. La idea es que las máquinas puedan analizar, entender y generar información que hablamos o escribimos.

Los documentos reales, ya sabes, tienen un montón de formas distintas de presentar la misma cosa: con abreviaturas, errores, a veces ni se entiende bien el formato, o lo dicen de mil maneras. Por eso, para procesar este lenguaje, se necesitan herramientas que capten el contexto y cómo se relacionan las distintas partes.

Aquí en ReceiptIA, el PLN se usa sobre todo en la etapa de entender lo que viene de una factura, y eso lo hace Gemini. De una factura, por ejemplo, sacamos nombres de negocios, lo que se compró, cuánto costó, fechas, y todo eso puede estar en cualquier sitio. El modelo ayuda a entender todo eso y a ponerlo en un formato ordenado. Entonces, el PLN es como un extra para el OCR. El OCR saca las letras que ve en una imagen, y el PLN se encarga de que entendamos qué significan esas letras.

4.5 Reconocimiento óptico de caracteres (OCR)

El OCR, o Reconocimiento Óptico de Caracteres, es una técnica que se usa para convertir texto de imágenes en texto digital que podemos manipular. Esto es súper útil cuando tienes documentos escaneados, fotos, formularios, tickets o facturas que quieres procesar. Básicamente, el sistema mira la imagen, detecta dónde están las letras y las convierte a texto. Con el OCR te ahorras tener que escribirlo todo a mano. Eso sí, la precisión puede variar dependiendo de cosas como la calidad de la imagen (resolución, luz, ángulos, ruido), lo bien impresa que esté la letra, el tamaño de las letras y qué tan complicado sea el documento.

En ReceiptIA, el OCR es el primer paso del procesamiento inteligente. Tú le das una foto de una factura y el sistema usa Tesseract para sacar el texto del documento. Tesseract es un programa de OCR gratuito y de código abierto. La gente que lo mantiene dice que la versión 5.x es la estable y que se puede usar su API para extraer texto de imágenes. Además, viene preparado para reconocer muchos idiomas y tipos de escritura.

Usar el OCR nos permite dividir el trabajo en dos partes: primero, sacamos la información de la imagen y, segundo, la interpretamos con inteligencia artificial. Esto hace que ReceiptIA esté mejor organizado y sea más fácil darnos cuenta de dónde pueden estar los fallos en cada paso.

4.6 Procesamiento digital de imágenes

El procesamiento digital de imágenes incluye varias técnicas que se usan para mirar, cambiar o mejorar imágenes con computadoras. Cuando se trata de reconocimiento óptico de caracteres, lo que tenga la imagen puede afectar mucho si el sistema puede reconocer bien las letras.

Una factura en una foto puede tener sombras, brillos, estar torcida, desenfocada o tener "ruido". Todo esto puede hacer que reconocer el texto sea más difícil y que la información que se saca no sea correcta.

La resolución es muy importante. Si las letras son muy chicas o no se ven claras, el sistema OCR podría confundir unas letras con otras que se parecen. Por eso, la calidad de las imágenes que entran es algo que hay que pensar en cualquier sistema que procese documentos.

En ReceiptIA, la calidad del documento que llega puede afectar al principio del proceso. Si hay un error al reconocer el texto con OCR, ese error puede pasarse a lo que interprete el modelo de inteligencia artificial. Por eso, es clave que el sistema considere la calidad de la imagen como algo que influye mucho en lo que sale.

4.7 Procesamiento inteligente de documentos

El procesamiento inteligente de documentos (IDP) junta varias tecnologías para convertir documentos que no tienen una estructura fija o que son más o menos estructurados en datos que las computadoras puedan usar.

A diferencia de un OCR normal, que solo convierte imágenes en texto, el procesamiento inteligente de documentos añade pasos de clasificación, extracción, interpretación y validación.

Un proceso de IDP se puede ver más o menos así:

Documento → OCR → se extraen los datos → se interpretan → se validan → se guardan.

Esta técnica está directamente relacionada con ReceiptIA. El sistema no solo muestra el texto que saca de una factura, sino que lo usa para obtener información organizada que luego se puede analizar fácilmente.

Al mezclar OCR con inteligencia artificial, se pueden manejar documentos que vienen en diferentes formatos. Esto es súper útil para las facturas, porque cada negocio puede tener su propio diseño y cómo se ve todo puede variar mucho.

4.8 Extracción y estructuración de información

Extraer información útil de un montón de datos es básicamente encontrar lo que necesitas en un texto más grande. Cuando se trata de documentos, esto puede ser hallar nombres, fechas, números, cantidades de dinero y otras cosas clave para hacer algo en particular.

Una vez que tienes los datos, hay que ponerlos en orden para poder guardarlos y usarlos fácilmente. Formatos como el JSON te ayudan a organizar la información como si fueran pares de cosas y sus valores, lo que permite que distintas partes de una aplicación se comuniquen entre sí de forma ordenada.

ReceiptIA usa este método para convertir la información de las facturas en algo estructurado. La parte del servidor (el backend) toma lo que la inteligencia artificial procesó y lo organiza para que la interfaz lo pueda usar, guardarlo en Firebase o exportarlo más adelante.

Además, al tener todo organizado, se pueden hacer comprobaciones. Por ejemplo, el sistema puede revisar si están todos los campos que necesita, si los datos tienen el formato correcto o si se cumplen ciertas reglas.

4.9 Detección de anomalías

Detectar irregularidades significa darse cuenta de datos o comportamientos que se salen de lo normal. Que algo sea una irregularidad no significa que haya fraude o un error seguro; más bien, indica algo que quizás haya que revisar. En los informes financieros o de administración, hay varias cosas que pueden hacer que necesitemos mirar más de cerca. Por ejemplo, si falta información, si los números no cuadran, si hay errores en las cuentas o si los datos no cumplen ciertas reglas.

En ReceiptIA, eso de encontrar irregularidades sirve como una ayuda. El sistema mira toda la información que recoge y puede señalar cosas que el usuario debería chequear.

Piensa en esto como una ayuda más, no como una decisión final sobre impuestos. La inteligencia artificial a veces se equivoca, así que las cosas que detecta hay que revisarlas a mano.

4.10 Automatización de procesos administrativos

Automatizar procesos administrativos significa usar tecnología para bajar o quitar de plano el trabajo manual que se repite una y otra vez.

Cuando hablamos de cosas de facturas, esto puede incluir leer los papeles, pasar los datos a otro lado, organizarlos, buscarlos y preparar todo para que entre en otros sistemas.

Con la automatización, se puede gastar menos tiempo en estas labores. Así, la gente puede dedicarse a cosas más importantes, como analizar y decidir.

ReceiptIA hace justo eso: automatiza la lectura y organización de los datos de las facturas. La idea no es que no intervenga nadie, sino dar una herramienta que quite trabajo repetitivo y haga más fácil revisar todo.

4.11 APIs REST

Una API, que significa Interfaz de Programación de Aplicaciones, es básicamente lo que permite que diferentes partes del software hablen entre sí, usando unas reglas y estructuras ya establecidas. Las APIs REST, en particular, siguen los principios de la arquitectura REST y suelen usar HTTP para comunicarse.

Los métodos HTTP que más se usan son, por un lado, GET, que sirve para obtener información; por otro, POST, para enviar o crear algo; luego están PUT o PATCH, que se usan para modificar recursos, y finalmente DELETE, para borrarlos.

Las respuestas que da una API también vienen con códigos de estado HTTP. Por ejemplo, si ves el código 200, significa que todo salió bien. Si los códigos están en el rango 400, es que hubo algún problema con tu solicitud, y si son de la categoría 500, es que el servidor tuvo un error.

En ReceiptIA, la API REST funciona como un intermediario entre lo que ves en pantalla (el frontend) y lo que hace el sistema por detrás (el backend).

El frontend envía una petición al servidor, y el backend se encarga de hacer lo que sea necesario para procesar el documento y devolver los resultados. Usar una API ayuda a que las tareas de la interfaz y la lógica de procesamiento estén separadas.

4.12 FastAPI

FastAPI es un framework de Python que se usa para crear interfaces de programación de aplicaciones. Tiene cosas como compatibilidad con OpenAPI, genera documentación por sí solo, valida datos, y maneja seguridad y autenticación. Algo muy bueno de FastAPI es que trabaja con Pydantic para crear y validar los modelos de datos. Con esto puedes decir qué tipo de información esperas recibir y enviar en tu aplicación. ReceiptIA usa FastAPI para su backend. Por medio de sus endpoints, recibe los documentos, los procesa y manda los resultados al frontend. Además, FastAPI te da documentación interactiva de la API usando OpenAPI, lo que hace más fácil probar y entender qué servicios hay.

4.13 Validación de datos y Pydantic

Validar datos significa comprobar que la información que llega a un sistema cumple con ciertas condiciones antes de que se procese.

Si una aplicación recibe información sin ninguna validación, puede fallar porque los tipos de datos no sean los correctos, falten campos, haya parámetros inválidos o aparezcan formatos inesperados.

Pydantic te permite definir modelos de datos y hace las validaciones por ti automáticamente. Cuando se usa con FastAPI, esto facilita mucho la creación de los "contratos" para lo que entra y lo que sale del sistema.

En ReceiptIA, la validación crea una estructura uniforme para toda la información que maneja el backend. Esto ayuda a equivocarse menos y hace que sea más fácil conectar las distintas partes del sistema.

4.14 Bases de datos y Firebase Firestore

Una base de datos ayuda a guardar la información de manera organizada para poder consultarla y modificarla más adelante. Hoy en día, en las aplicaciones, usamos diferentes tipos de almacenamiento, como las bases de datos relacionales y las NoSQL.

Las bases de datos NoSQL son más flexibles que las bases de datos relacionales de toda la vida. Firestore, que es lo que usa ReceiptIA, permite guardar datos en documentos y colecciones.

Esta manera de organizar las cosas va muy bien para datos que van creciendo poco a poco y que necesitan ser accedidos por diferentes partes de una aplicación.

En ReceiptIA, usamos Firebase para guardar los datos de los usuarios, las facturas, el historial y para organizar los documentos. Al usar un servicio que ya está administrado, no tenemos que construir todo el sistema de almacenamiento y la gestión de servidores desde cero.

4.15 Autenticación y autorización

La autenticación es básicamente el proceso con el que un sistema verifica quién eres. En cambio, la autorización es lo que te permite hacer una vez que ya te ha validado. Es importante entender esta diferencia, sobre todo en aplicaciones donde hay información que no todo el mundo debería ver.

ReceiptIA usa Firebase Authentication para controlar quién entra. Sabiendo quién es cada usuario, se pueden asignar permisos y roles para ver qué cosas puede usar cada uno.

Así se protege la información guardada en la app y es una parte fundamental de la seguridad general del sistema.

4.16 Docker y contenedorización

Con la técnica de contenedorización, puedes juntar una aplicación con todo lo que necesita, sus requisitos y configuraciones, para que funcione igual en cualquier lado. Docker usa algo llamado "imágenes" y "contenedores" para esto.

Una imagen tiene todo lo necesario para crear un lugar donde correr la aplicación, y un contenedor es básicamente una ejecución de esa imagen. Lo mejor de usar contenedores es que se reducen un montón los problemas cuando las cosas no funcionan igual en diferentes sitios. A veces, una app va bien en tu computadora, pero luego falla en otra porque las librerías o lo que sea que necesite está en otra versión.

En ReceiptIA, usamos Docker para que el backend y todo lo que necesita para funcionar, incluido lo del OCR, sea más fácil de manejar. Esto hace que el proyecto sea más fácil de mover de un lado a otro y de replicar.

4.17 Integración continua

La integración continua, o CI por sus siglas en inglés, es una forma de trabajar en el desarrollo de software donde se van metiendo y revisando los cambios en un proyecto muy a menudo. Lo importante de esto es pillar los fallos cuanto antes y asegurarnos de que lo que cambiamos no fastidia cómo funciona la aplicación.

Cuando se usa integración continua, los cambios que se suben al sitio donde se guarda el código pueden hacer que arranquen unas rutinas automáticas. Estas rutinas se encargan de instalar lo que haga falta, probar el código, revisarlo y ver si cumple con todo lo que el equipo de desarrollo ha decidido. Así, se necesita menos gente revisando a mano y el proyecto se valida todo el rato.

En ReceiptIA, usamos GitHub Actions para hacer una parte de esto de forma automática. Lo que hemos montado hace que se instalen las librerías del proyecto, se pasen herramientas que analizan el código sin ejecutarlo y se corran pruebas automáticas con Pytest. De esta manera, cada cambio se puede comprobar con varias cosas antes de que se meta de lleno en el proyecto. Con la integración continua, ReceiptIA es mejor y más fácil de mantener. Permite encontrar problemas pronto y da más seguridad cuando se cambia el código.

4.18 Observabilidad y logging

La observabilidad básicamente es saber cómo está funcionando una aplicación por dentro, basándose en lo que va generando mientras trabaja. Para conseguir esto, solemos usar logs, métricas y trazabilidad. Los registros, o logs, se usan para guardar información importante sobre cosas que pasaron mientras el sistema corría. Estos registros nos sirven para pillar errores, ver cuánto tardan las cosas en responder y entender mejor cómo van ciertas operaciones.

En ReceiptIA, pusimos en marcha un sistema de observabilidad en el backend usando un middleware HTTP. Esto documenta datos de las peticiones, como a dónde va la petición, qué método se usó, qué respuesta dio, cuánto tardó, qué modelo de IA se usó y si hubo algún error.

La observabilidad es súper importante en ReceiptIA, sobre todo porque dependemos de cosas de fuera, como el modelo de IA. Los logs nos ayudan a darnos cuenta si un fallo viene del propio backend, de cómo se procesan los datos o de alguna dependencia externa. También pensamos en la seguridad de los datos. Es clave que los registros no guarden información privada de más, como detalles de facturas, contraseñas o tokens de acceso.

4.19 Identificación de solicitudes mediante request_id

El request_id es un identificador único que se asocia a una petición hecha a una aplicación. Sirve principalmente para poder agrupar distintos eventos que son parte de una misma acción.

En sistemas que procesan muchas peticiones, tener un identificador ayuda a diferenciar los registros de cada usuario o acción. Esto es útil para buscar errores y para seguir el rastro de una solicitud, desde que empieza hasta que se da una respuesta.

ReceiptIA genera un request_id para las peticiones que maneja el backend y lo pone en los logs para que se puedan monitorear. Así, si ocurre un error o una respuesta tarda más de lo esperado, se pueden encontrar todos los eventos relacionados con esa petición. Este mecanismo mejora la trazabilidad del sistema y es algo importante para diagnosticar y mantener aplicaciones que usan varios componentes y servicios externos.

4.20 Monitoreo y Health Checks

La supervisión consiste en estar mirando de cerca cómo funciona un sistema y si está accesible, pero de forma regular. El objetivo es pillar cualquier problema que pueda afectar al servicio y tener datos para poder actuar rápido. Algo que se hace mucho es poner en marcha chequeos de "salud", que básicamente es mandar una petición a un sitio específico para ver si un servicio está en marcha y va bien. ReceiptIA tiene ese sitio /health, y lo usamos para ver cómo está el backend. Además, hemos montado algo automático con GitHub Actions que revisa este sitio cada cierto tiempo. Todo esto de la supervisión viene a sumar a las pruebas automáticas que hacemos cuando estamos desarrollando. Las pruebas sirven para

asegurarnos de que las cosas funcionan como deben, y la supervisión se encarga de que el servicio siga estando ahí cuando ya está funcionando.

4.21 Rendimiento y benchmarking

El rendimiento de una aplicación es básicamente qué tan bien puede manejar las peticiones que recibe. Para medirlo, se usan cosas como cuánto tarda en responder, qué tan rápido llega la respuesta, cuántas peticiones puede procesar y con qué frecuencia falla.

Hacer un "benchmarking" es básicamente poner a prueba un sistema bajo condiciones controladas para ver cómo se comporta. Esto nos da datos para analizarlo mejor.

Para ver cuánto tardan las respuestas, también se usan los percentiles. El P50 te dice el tiempo que tarda la mitad de las peticiones, más o menos. El P95 te da una idea de cómo va la gran mayoría de las peticiones, y te ayuda a ver si hay algunas que se quedan atascadas mucho tiempo.

En ReceiptI/A hicimos una prueba de rendimiento en el punto principal donde se procesan las facturas. Así pudimos ver cuánto tiempo nos toma procesar cada petición y si teníamos problemas por depender de un servicio externo de inteligencia artificial.

Todo esto es importante porque nos deja saber cómo funciona realmente el sistema con datos de verdad, y así podemos pensar en cómo hacerlo mejor, que responda más rápido, que no falle y que maneje mejor los errores.

4.22 Seguridad de aplicaciones

La protección de aplicaciones se refiere a las medidas que se toman para mantener seguro un sistema, sus usuarios y la información que maneja, protegiéndolos de accesos no permitidos, pérdida de datos o uso indebido.

Para una aplicación como ReceiptI/A, la seguridad es especialmente importante porque maneja documentos que pueden tener tanto información de negocios como datos personales. Por eso, es clave controlar quién accede a las cosas y proteger bien las contraseñas y claves que usan los diferentes servicios.

Algo básico que se hace es no escribir las contraseñas directamente en el código. ReceiptI/A usa variables de entorno para configurar cosas y guardar credenciales, y el archivo .env.example se usa para mostrar qué variables se necesitan, pero sin poner los valores reales.

También se usa Firebase para la autenticación y autorización, además de revisar los archivos que llegan y limitar la información que se guarda en los logs. Hay que ver la seguridad como algo que se hace todo el tiempo, no solo al final del proyecto.

4.23 Limitaciones y riesgos de la inteligencia artificial

Los sistemas de inteligencia artificial, para que se entienda bien, a veces se equivocan y no hay que pensar que son perfectos. Lo que hagan o dejen de hacer depende mucho de cómo sean los datos que les metemos, de cómo estén los documentos, de las instrucciones que les demos y de las condiciones del servicio que estemos usando.

Aquí en ReceiptIA, si la foto no se ve bien, es normal que el reconocimiento óptico de caracteres (OCR) falle. Como luego la IA usa ese texto que saca el OCR, si al principio ya hay un error, arrastra problemas para lo que venga después.

También pasa que la IA se haga un lío con algunos datos o suelte algo que no cuadra del todo con lo que pone en el papel. Por eso, lo que saque hay que verlo como una ayuda, y a veces alguien tiene que revisarlo.

Otra cosa es que dependemos de otros servicios. ReceiptIA usa Gemini a través de una API, así que cosas como si está disponible, los límites de uso, las cuotas o cuánto tarda en responder, todo eso puede afectar cómo funciona.

Hay que tener en cuenta estas limitaciones cuando miremos los resultados de esto y pensemos en cómo mejorarlo en el futuro.

4.24 Relación de los conceptos con ReceiptIA

Con los principios que se presentan aquí, se entiende mejor la tecnología detrás de ReceiptIA. La inteligencia artificial y los modelos generativos son los que permiten analizar la información, y el OCR es el que toma lo que se ve en las facturas y lo convierte en texto que se puede usar. Las APIs REST y FastAPI se encargan de que el frontend y el backend se comuniquen, y Pydantic se usa para asegurar que los datos sean correctos.

Firebase se ocupa de la autenticación y de guardar cosas, y con Docker el backend se ejecuta sin problemas junto con todo lo que necesita.

Además, las pruebas automáticas, la integración continua y el análisis estático hacen que el código sea de mejor calidad. La observabilidad, usar un "request_id", hacer benchmarking y el monitoreo nos ayudan a ver cómo funciona todo y a pillar los problemas antes de que se hagan grandes.

Todo esto junto hace que ReceiptIA sea una herramienta completa para procesar facturas de forma inteligente, combinando IA con buena forma de desarrollar, seguridad, pruebas y funcionamiento del software.

5. Análisis del problema

Analizar y gestionar facturas es algo que se hace todo el tiempo en empresas, comercios, contabilidad y auditorías. Estos papeles tienen información clave para apuntar compras, ventas, gastos y todo tipo de transacciones. Por eso, si no se interpretan y organizan bien, los registros administrativos no van a ser confiables.

Pero claro, si uno procesa todo a mano, tiene que mirarse cada factura, buscar los datos importantes y luego meterlos o pasarlos a otro sistema. Si tienes un montón de facturas, esto se puede volver súper aburrido. Aparte de que lleva tiempo, al hacerlo manualmente puedes cometer errores al copiar datos, saltarte cosas o tener problemas para mantener todo bien organizado.

Otro lío viene cuando intentas encontrar cosas raras. Una factura puede tener números que no cuadran, cálculos mal hechos, datos que faltan o cosas que necesiten que alguien les eche un segundo vistazo. Si todo esto depende de que una sola persona lo revise, se hace más lento y depende mucho de lo que sepa esa persona.

Básicamente, el problema que vieron es este: la gente saca los datos a mano, cuesta revisar muchos papeles, la organización de lo que se guarda es limitada y hace falta detectar fallos más rápido.

Frente a todo esto, se ve que la inteligencia artificial podría ayudar bastante. La idea no es quitarle el trabajo a la gente del todo, sino automatizar lo que es más pesado y darle al usuario información ordenada para que tome mejores decisiones y pueda revisar las cosas más fácilmente después.

ReceiptIA se encarga de esta situación mediante un proceso automático. El usuario manda una foto de la factura y el sistema usa OCR para sacar el texto. Luego, este texto pasa por un modelo generativo que reconoce los datos importantes y arma una versión organizada de la información. Con estos datos, también se pueden usar reglas para ver si hay algo raro.

El problema se puede ver así:

Revisión a mano → lleva más tiempo → se pueden cometer errores → es difícil tener todo ordenado → se gasta más tiempo en trámites.

La solución que proponemos busca cambiar esto a: Subir el documento → OCR → análisis con IA → validación → detectar cosas raras → guardar → revisar y sacar.

Así que, el problema principal que ReceiptIA quiere arreglar es lo complicado que es procesar y tener bien organizada la información de las facturas rápido y de forma clara, ofreciendo una herramienta que haga parte del trabajo sola y ayude a encontrar los datos clave y posibles fallos. Es importante saber que el sistema no pretende decir si una factura es válida fiscalmente por sí solo. Lo que genera la inteligencia artificial es una ayuda para el usuario y puede necesitar que alguien lo revise, sobre todo si las fotos no se ven bien, la información no está clara, los documentos tienen formatos raros o el modelo se equivoca. Por eso, el tema no se

mira solo desde el lado de la inteligencia artificial, sino que se presenta una solución completa que incluye desde el análisis de las imágenes, la obtención de datos, lo que hace la IA, el almacenamiento, la comprobación de quién es quién, la automatización de revisiones, hasta el seguimiento y la vigilancia.

6. Propuesta de solución

El equipo propone la creación de ReceiptIA, una plataforma web dedicada a la gestión inteligente de facturas y comprobantes, empleando reconocimiento óptico de caracteres y tecnologías de inteligencia artificial.

La solución se ha organizado mediante una arquitectura donde cada componente cumple una función específica. La interfaz de usuario gestiona la interacción, mientras que el sistema de backend se encarga del procesamiento y coordina los servicios de OCR, inteligencia artificial y almacenamiento de datos.

El objetivo principal de esta solución es automatizar parte del proceso de lectura y análisis de facturas, permitiendo extraer información relevante, identificar posibles anomalías y guardar los resultados para su consulta y revisión posteriores.

6.1 Funcionamiento general de la solución

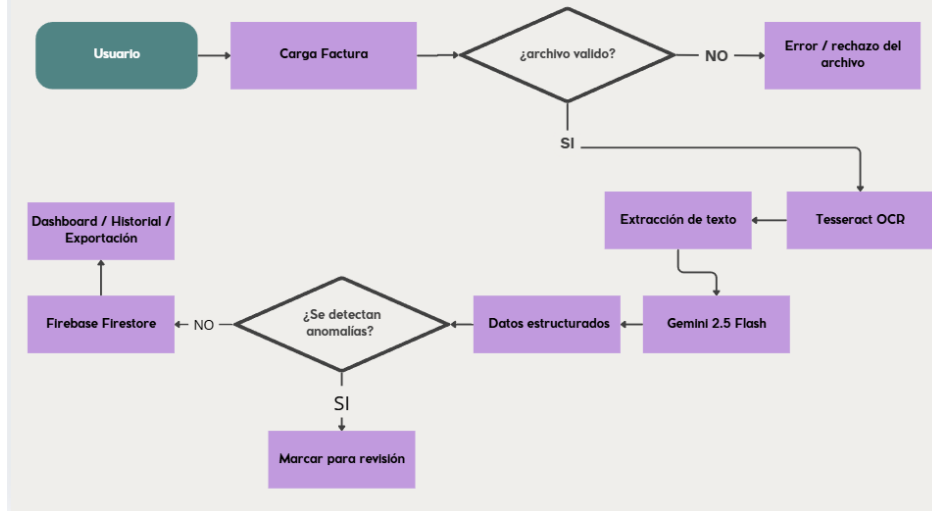
El proceso de ReceiptIA se inicia cuando el usuario accede a su cuenta y elige una factura o recibo desde la plataforma en línea. El archivo es transmitido al servidor backend construido con FastAPI a través de una solicitud HTTP.

Tras la recepción del documento, el backend realiza las validaciones necesarias y luego emplea Tesseract OCR para extraer el texto presente en la imagen. El texto resultante sirve como entrada para Gemini 2.5 Flash, que se encarga de analizar la información y organizar los datos esenciales de la factura.

Entre los datos que se pueden recolectar se incluyen el comercio, la fecha, el NIT, los productos o servicios, el subtotal, el IVA y el total. A continuación, el backend procesa la respuesta, ejecuta las correspondientes validaciones y evalúa si hay posibles irregularidades que requieran atención.

Finalmente, los resultados se presentan al usuario a través de la interfaz y tienen la opción de ser almacenados en Firebase Firestore, lo que permite mantener un expediente de los documentos procesados. La aplicación facilita también la organización de la información y la exportación de ciertos resultados para su uso en Excel.

FLUJO DE PROCESAMIENTO DE UNA FACTURA EN RECEIPTIA



Se puede ver cómo fluye el proceso principal para tratar una factura en ReceiptIA. Todo empieza cuando un usuario sube un archivo y el sistema verifica que sea un documento válido. Si pasa esa revisión, Tesseract OCR se encarga de sacar todo el texto de la factura. Después, Gemini 2.5 Flash analiza ese texto para organizar la información. Los datos que se consiguen se revisan para ver si hay algo raro. Si se detectan problemas, el documento se marca para que alguien lo mire más a fondo. Luego, el resultado se guarda en Firebase Firestore y se puede ver desde el Dashboard, junto con su historial, o incluso exportarlo. Si el archivo no cumple con lo que se pide, el sistema simplemente no lo procesa.

6.2 Componentes principales

La solución se compone de varios componentes que operan de manera conjunta para llevar a cabo el procesamiento de las facturas.

Interfaz de usuario: La interfaz de usuario ofrece el medio a través del cual el usuario se relaciona con ReceiptIA. A partir de esta interfaz, se pueden llevar a cabo acciones como la autenticación, la carga de facturas, la visualización de los resultados, la consulta del historial, la organización de documentos y la exportación de datos.

Componente servidor: El componente servidor fue construido utilizando FastAPI y representa el núcleo de la solución. Su función es recibir las solicitudes del componente de usuario, validar los archivos, coordinar el procesamiento mediante OCR y técnicas de inteligencia artificial, procesar las respuestas generadas y comunicarse con los servicios de almacenamiento.

Reconocimiento óptico de caracteres: Se utiliza Tesseract OCR para convertir la información visual de las facturas en texto. Esta fase permite obtener el contenido que será posteriormente analizado por el modelo de inteligencia artificial. Análisis

por inteligencia artificial: Gemini 2.5 Flash es el encargado de interpretar el texto extraído mediante OCR y organizar la información pertinente de la factura. También contribuye al análisis de ciertos datos con el fin de detectar posibles irregularidades.

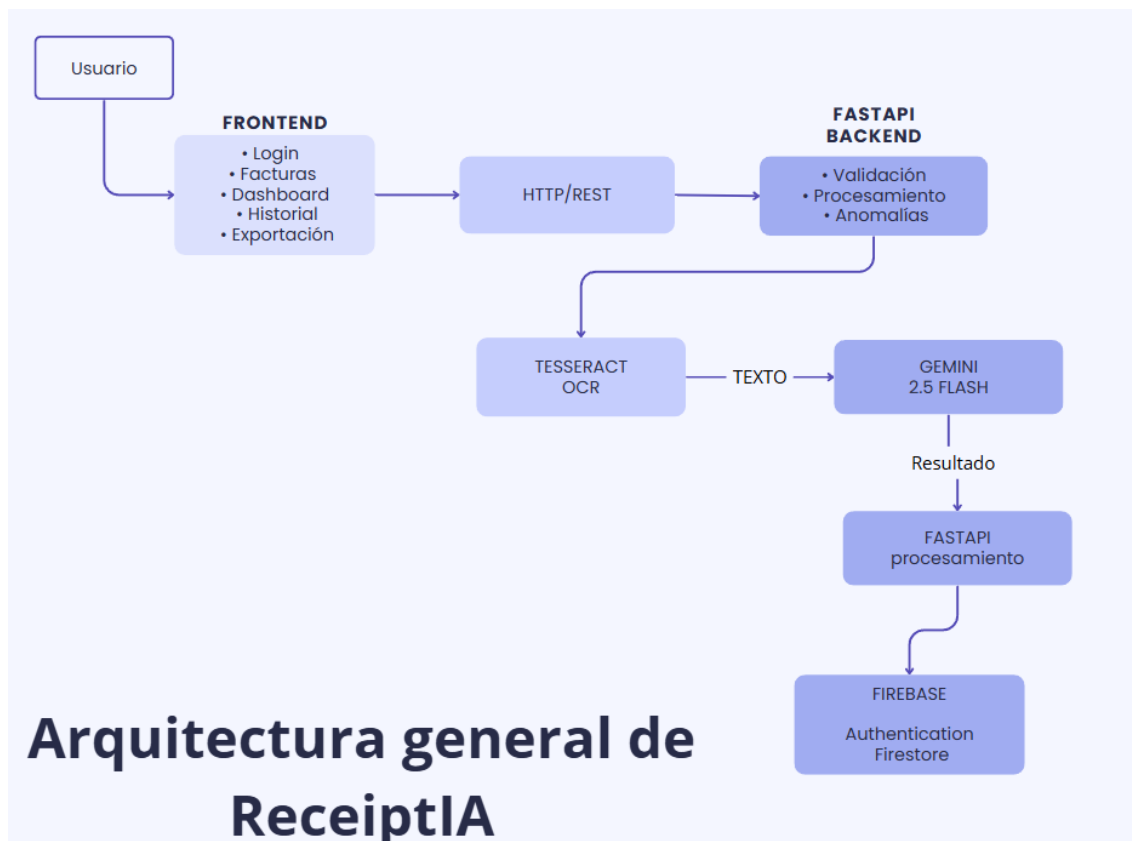
Gestión de autenticación: Firebase Authentication se encarga de administrar la autenticación de los usuarios y regular el acceso a la aplicación. Almacenamiento en Firestore: Firestore facilita el almacenamiento de información asociada con los documentos procesados y la conservación de un registro que se puede consultar más adelante.

Exportación de datos: La aplicación posibilita la conversión de ciertos resultados obtenidos del procesamiento a un formato que puede ser exportado y utilizado en Excel, lo que facilita su integración en otros procedimientos administrativos.

6.3 Arquitectura y diseño de la solución

El diseño de ReceiptIA divide las tareas entre la interfaz, el procesamiento del lado del servidor, los servicios de inteligencia artificial y las bases de datos. Esta separación facilita que cada parte evolucione independientemente y hace que la administración del sistema sea más sencilla. La interfaz gráfica se comunica con el backend mediante peticiones HTTP.

FastAPI funciona como intermediario entre la interfaz y los distintos servicios que se utilizan para procesar la información. El backend es responsable de coordinar la comunicación con Tesseract OCR para extraer el texto y, posteriormente, utiliza Gemini 2.5 Flash para comprender y estructurar los datos. Una vez que la información ha sido procesada, los resultados se pueden almacenar usando Firebase y luego mostrarlos de nuevo al usuario.



Se muestra cómo está organizada ReceiptIA en general y cómo se relacionan sus partes importantes. El usuario interactúa con la parte del frontend, y esta, a su vez, se comunica vía HTTP/REST con el backend, que está hecho con FastAPI. El backend usa Tesseract OCR y Gemini 2.5 Flash para procesar todo. Una vez que se consigue y se maneja la información, FastAPI se encarga de gestionar los resultados. Además, usa Firebase para verificar la identidad de los usuarios y guardar los datos en Firestore.

6.4 Decisiones técnicas

Las tecnologías que usamos las elegimos basándonos en lo que el proyecto necesitaba, tanto en funciones como en aspectos técnicos. Para el backend, nos quedamos con FastAPI porque trabaja bien con Python, es bueno para crear APIs REST y usa Pydantic para validar cosas.

Además, te da una documentación con la que puedes interactuar para probar los endpoints, lo que hace que probar y desarrollar sea más fácil. Para reconocer el texto en las imágenes de las facturas, usamos Tesseract OCR. Esto nos deja convertir esas imágenes en texto que luego puede usar la inteligencia artificial.

Como componente de inteligencia artificial, elegimos Gemini 2.5 Flash. Nos pareció el mejor por cómo analiza texto y organiza la información. Con él, podemos procesar facturas, aunque tengan formatos o distribuciones de datos diferentes. Firebase se encargó de la autenticación y el almacenamiento. Al usarlo, nos ahorramos tener que crear nosotros mismos un sistema de autenticación y de guardar datos desde cero, ya que ya venían especializados.

Para que el entorno del backend y todo lo que necesita funcionara igual en cualquier sitio y fuera fácil de repetir, usamos Docker.

Y para que las pruebas, validaciones y el monitoreo se hicieran solos, implementamos GitHub Actions. Esto ayudó a que el proyecto tuviera mejor calidad y fuera más fácil de mantener.

6.5 Justificación de la solución

Integrar OCR e inteligencia artificial ayuda a automatizar varias partes del proceso de manejo de documentos. Tesseract OCR extrae el texto de la factura, y luego Gemini 2.5 Flash se encarga de entender esa info y ponerla en un formato ordenado.

Usar una arquitectura separada también tiene el beneficio de mantener las diferentes partes del sistema trabajando de forma independiente. El frontend se centra en cómo lo usa el usuario, y el backend se ocupa de manejar el procesamiento y hablar con servicios de afuera.

Con esta arquitectura, hacer cambios en el futuro es mucho más fácil. Por ejemplo, se podría cambiar o mejorar el módulo OCR con otra tecnología sin tener que rehacer todo el frontend. De la misma manera, si surgen nuevas necesidades de precisión, costos o rendimiento, el módulo de IA podría ser reemplazado por otro modelo.

Además, Firebase simplifica la administración de usuarios y guardar la información que se procesa. Y con Docker, GitHub Actions y sistemas de monitoreo, se complementa la solución con métodos que se enfocan en la calidad, el mantenimiento y el funcionamiento del sistema.

Por todo esto, la idea no es solo hacer una herramienta que lea facturas, sino crear una solución completa que junte el procesamiento de documentos, inteligencia artificial, almacenamiento, autenticación y buenas prácticas de desarrollo de software.

7. Metodología de desarrollo del proyecto o investigación

El desarrollo de ReceiptIA se dio paso a paso, añadiendo poco a poco las funcionalidades que hacían falta para solucionar el problema que teníamos. Empezamos definiendo bien qué necesitábamos y cuál sería la estructura principal. Después vino el trabajo de procesar los documentos, la parte de inteligencia artificial, el almacenamiento de datos, la autenticación de usuarios, las pruebas y el sistema para vigilarlo todo.

El proceso de creación se dividió en varias etapas: primero entendimos el problema y los requisitos. Luego diseñamos cómo iba a ser la solución. Después creamos el backend y el frontend. Integramos la tecnología OCR y la inteligencia artificial. Establecimos el sistema de autenticación y el almacenamiento. Hicimos pruebas y validaciones. Luego pasamos a la contenedorización, el monitoreo, la supervisión y, por último, la evaluación final.

7.1 Análisis y definición de requerimientos

En la primera etapa se identificaron las principales dificultades relacionadas con el procesamiento manual de facturas y se establecieron las funcionalidades que debía cumplir la solución.

- Entre los requerimientos definidos se encontraron:
- Carga de imágenes de facturas y recibos.
- Extracción automática de información.
- Procesamiento mediante OCR.
- Interpretación mediante inteligencia artificial.
- Identificación de posibles anomalías.
- Autenticación de usuarios.
- Almacenamiento de información.
- Historial de documentos.
- Organización de facturas mediante carpetas.
- Visualización mediante Dashboard.
- Exportación de información a Excel.
- Pruebas y validación del funcionamiento.

A partir de estos requerimientos se establecieron los componentes tecnológicos necesarios y se definió una arquitectura que permitiera separar las diferentes responsabilidades de la aplicación.

7.2 Diseño de la arquitectura

Una vez que se establecieron los requisitos, se elaboró la estructura de la aplicación y se determinó la función de cada uno de sus elementos.

El frontend fue concebido como el área de interacción del usuario, mientras que FastAPI fue designado como el componente que centraliza la lógica del backend. Se integró Tesseract OCR para el proceso de extracción de texto y Gemini 2.5 Flash para la interpretación de datos.

Firebase fue elegido para ofrecer servicios de autenticación y almacenamiento, y se añadieron Docker y GitHub Actions posteriormente como herramientas destinadas a la portabilidad, automatización y mejora de la calidad del proyecto.

7.3 Desarrollo del backend

El desarrollo del backend se llevó a cabo empleando FastAPI, configurándose como el elemento fundamental responsable de gestionar las peticiones enviadas por el frontend y supervisar el tratamiento de los documentos.

Se crearon puntos finales para llevar a cabo las diversas funciones de la aplicación, abarcando el manejo de facturas. Uno de los puntos finales esenciales corresponde a:

POST /procesar

Este último punto es una parte fundamental del sistema, ya que permite empezar a procesar la información que el usuario nos proporciona.

```
@app.post("/procesar")
async def procesar(file: UploadFile = File(...)):
    try:
        # =====
        # VALIDAR EL ARCHIVO
        # =====

        if file.content_type not in [
            "image/jpeg",
            "image/png"
        ]:
            raise HTTPException(
                status_code=400,
                detail="Solo se permiten imágenes JPG o PNG."
            )
```

la implementación del endpoint POST /procesar mediante FastAPI. El endpoint recibe el archivo enviado por el usuario y realiza inicialmente una validación del tipo de contenido, permitiendo únicamente imágenes en formato JPG o PNG. En caso de recibir un formato no permitido, el sistema genera una respuesta de error

HTTP 400. Esta validación permite controlar las entradas antes de iniciar las etapas posteriores de procesamiento.

7.4 Implementación del reconocimiento óptico de caracteres

Después de tener listo el backend, se añadió Tesseract OCR para que se encargara de sacar el texto de las imágenes de las facturas.

El sistema toma el archivo que manda el usuario y usa el motor OCR para transformar lo que se ve en la imagen en texto. Luego, este texto se usa como entrada para el módulo de inteligencia artificial.

```
if not TESSERACT_CMD_DETECTADO:
    raise RuntimeError(
        "No se encontró Tesseract OCR. "
        "Ejecuta Instalar_Tesseract.bat o configura TESSERACT_CMD en el archivo .env."
    )

try:
    texto = pytesseract.image_to_string(imagen_limpia, lang="spa+eng")
except pytesseract.TesseractNotFoundError:
    raise RuntimeError(
        "Tesseract OCR no fue encontrado. "
        "Instálalo o configura TESSERACT_CMD en el archivo .env."
    )
except pytesseract.TesseractError:
    texto = pytesseract.image_to_string(imagen_limpia, lang="eng")

return texto
```

la implementación del reconocimiento óptico de caracteres mediante Pytesseract. El sistema utiliza la función `image_to_string()` para convertir la imagen procesada en texto. Se contempla además el manejo de errores relacionados con la ausencia de Tesseract y se establece el idioma de reconocimiento, utilizando español como primera opción y una alternativa en inglés cuando se produce un error durante el procesamiento.

7.5 Integración de inteligencia artificial

Una vez que se obtiene el texto con OCR, se usa Gemini 2.5 Flash como la IA para revisar los datos y armar una respuesta. El backend acomoda la info que se le manda al modelo y ajusta la configuración para controlar cómo procesa todo.

Para poder seguir mejor el proceso, se anota cuánto tiempo tarda el modelo en cada pedido. Así se puede ver cómo se relaciona el uso de la inteligencia artificial con las métricas de rendimiento y la visibilidad que se consiguen al evaluar el sistema.


```

client = obtener_cliente_gemini()

inicio_gemini = time.perf_counter()

try:
    response = client.models.generate_content(
        model="gemini-2.5-flash",
        contents=prompt,
        config=GEMINI_CONFIG,
    )

    duracion_gemini_ms = round(
        (time.perf_counter() - inicio_gemini) * 1000,
        2
    )

    print(f"Gemini completado en: {duracion_gemini_ms} ms")

```

Se presenta la invocación efectuada al modelo a través de `client.models.generate_content()`. Se indica `gemini-2.5-flash` como el modelo aplicado y se transmiten el contenido producido mediante el aviso y la configuración asociada. Adicionalmente, se emplea `time.perf_counter()` para evaluar el tiempo que toma el procesamiento realizado por Gemini, lo que permite que esta información sea registrada más tarde para propósitos de observación y análisis del rendimiento.

7.6 Implementación de autenticación y almacenamiento

Para la gestión de usuarios y el almacenamiento de información, usamos Firebase Authentication y Firebase Firestore. Firebase Authentication se encarga de controlar quién entra a la aplicación usando sus datos de acceso, y Firestore es el sistema donde guardamos los datos de las facturas y el historial de lo que pasa en la app.

La parte de autenticación se hizo aprovechando lo que da Firebase Authentication, para que la gente se pueda registrar, entrar y salir, y que la app recuerde si ya iniciaste sesión.

En cuanto a guardar datos, la aplicación tiene unas funciones que toman la información de las facturas que se manejan y la mandan a Firestore. Así, los datos se quedan guardados para poder verlos después cuando se necesiten.

```

await setPersistence(auth, persistencia);

const credencial = await signInWithEmailAndPassword(auth, email, password);

```

Para iniciar sesión, la aplicación usa Firebase Authentication. Con el correo y la contraseña que el usuario pone, se autentica con la función

signInWithEmailAndPassword. Así, se obtienen las credenciales necesarias para que el usuario siga conectado.

```
await guardarFacturaEnFirestore(facturaHistorial);

if(factura.tiene_anomalias && factura.anomalias && factura.anomalias.length > 0){
    const esErrorOCR = factura.anomalias.some(a =>
        String(a.tipo || '').toLowerCase().includes('error') ||
        String(a.tipo || '').toLowerCase().includes('texto no detectado')
    );

    if(esErrorOCR){
        errores++;
    }else{
        procesadas++;
    }
}else{
    procesadas++;
}
```

Esto muestra cómo la información de una factura que ya se procesó se envía a la función guardarFacturaEnFirestore. Esa función se encarga de guardarla en Firestore. Así, los resultados que se obtuvieron al procesar la factura se quedan guardados y luego es más fácil encontrarlos desde el historial de ReceiptIA.

7.7 Implementación de pruebas automatizadas

Durante el desarrollo del proyecto, realizamos pruebas automáticas con Pytest. Estas pruebas nos ayudaron a comprobar diferentes aspectos del backend y a encontrar errores que surgían mientras trabajábamos.

Al usar pruebas automáticas, pudimos volver a ejecutar las validaciones cada vez que hacíamos cambios en el código, lo que redujo bastante la posibilidad de que algo ya funcionara mal.

Además, incorporamos Ruff para hacer un análisis estático. Lo usamos para encontrar fallos en el código y así mantener un nivel de calidad determinado.

```
def test_health_responde_correctamente():
    """Comprueba que el backend esté disponible."""

    response = client.get("/health")

    assert response.status_code == 200
    assert response.json() == {
        "status": "ok",
        "servicio": "ReceiptIA Backend",
    }
```

Hace una comprobación automática para ver si el sistema backend está funcionando bien, usando el endpoint /health. Lo que se comprueba es que la respuesta dé un código HTTP 200 y que lo que llega coincida con lo esperado. Esto ayuda a validar el estado del servicio automáticamente.

```
platform win32 -- Python 3.12.10, pytest-9.1.1, pluggy-1.6.0
rootdir: C:\Users\Wilber JJR\OneDrive - Universidad Gerardo Barrios\Escritorio\ReceiptIA-feature-monitreo-evaluacion-ia
plugins: anyio-4.14.2
collected 11 items

Backend\tests\test_backend.py ..... [100%]

===== warnings summary =====
..\..\..\AppData\Local\Programs\Python\Python312\Lib\site-packages\fastapi\testclient.py:1
C:\Users\Wilber JJR\AppData\Local\Programs\Python\Python312\Lib\site-packages\fastapi\testclient.py
:1: StarletteDeprecationWarning: Using `httpx` with `starlette.testclient` is deprecated; install `ht
tpx2` instead.
    from starlette.testclient import TestClient as TestClient # noqa

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
===== 11 passed, 1 warning in 3.96s =====
```

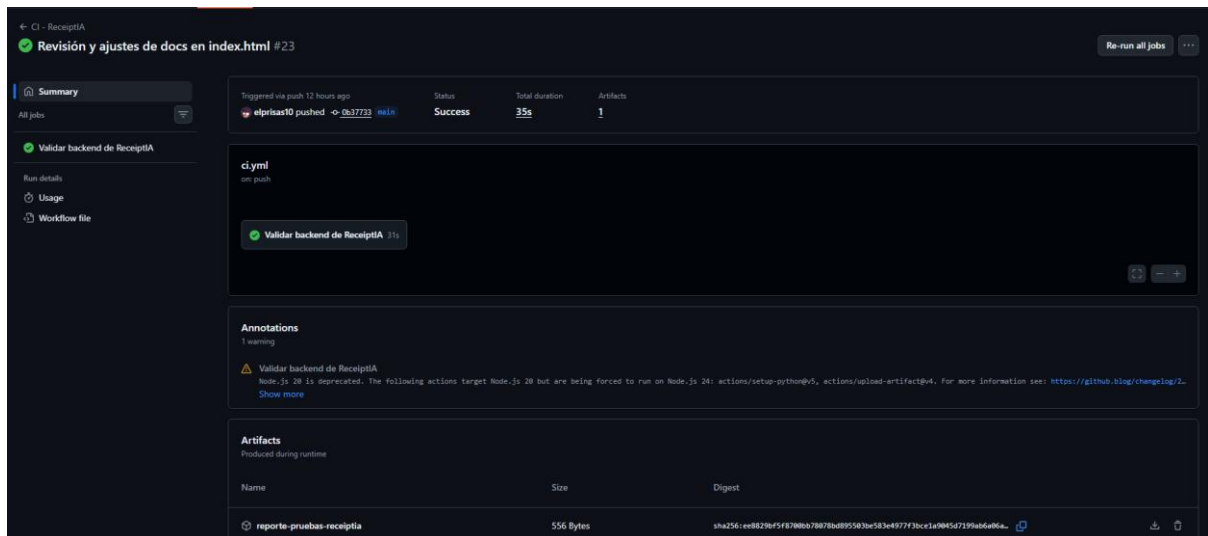
Se presenta la realización del conjunto de pruebas automatizadas del proyecto. Se reunieron un total de 11 pruebas, todas las cuales se llevaron a cabo con éxito, logrando un 100 % de eficacia en la ejecución. La duración de este procedimiento fue de alrededor de 3.96 segundos.

7.8 Integración continua mediante GitHub Actions

En ReceiptIA, hemos implementado un sistema de integración continua usando GitHub Actions, como parte de nuestras metodologías de desarrollo. Esta herramienta nos permite automatizar tareas relacionadas con la revisión del código y la ejecución de pruebas cada vez que se hacen cambios en el repositorio.

Con la integración continua, podemos asegurarnos de que el proyecto sigue funcionando bien después de aplicar modificaciones. Así, se reduce la posibilidad de que aparezcan errores que afecten funciones que ya estaban funcionando.

En ReceiptIA, usamos GitHub Actions para automatizar las comprobaciones que necesita el proyecto, como correr pruebas y otras validaciones de la calidad del código. Esto se suma a las pruebas manuales que hacemos con Pytest y crea un proceso de validación que se repite de forma automática.



Se observa que el flujo de trabajo CI - ReceiptIA se ha ejecutado de forma satisfactoria. Durante la demostración, el proceso de validación del backend finalizó correctamente, arrojando un estado de Éxito y demorando aproximadamente 35 segundos. Por otra parte, el flujo de trabajo generó el artefacto reporte-pruebas-receptia, que se utiliza para guardar la información relacionada con la ejecución de las pruebas.

7.9 Contenedorización mediante Docker

Para las actualizaciones que hicimos en la solución, pusimos Docker para que el backend funcionara mejor, junto con todo lo que necesita para andar bien.

Al meter todo en un contenedor, es más fácil juntar la aplicación y lo que no puede faltar, creando un ambiente que se puede copiar. Esto ayuda a que no haya tantas diferencias entre cómo funciona en tu computadora y cómo va en otros lados.

En ReceiptIA hicimos un Dockerfile. Ahí vienen las instrucciones para armar la imagen del backend. En ese archivo pusimos las librerías que usa la app y todo lo que se necesita para hacer el OCR.

Usar Docker hace que mover el proyecto sea más fácil y que sea simple crear más copias del ambiente de trabajo.

```

FROM python:3.12-slim

# Evita archivos .pyc
ENV PYTHONDONTWRITEBYTECODE=1

# Muestra inmediatamente los print()
ENV PYTHONUNBUFFERED=1

# Instalar Tesseract y librerías necesarias
RUN apt-get update && apt-get install -y \
    tesseract-ocr \
    tesseract-ocr-spa \
    tesseract-ocr-eng \
    libgl1 \
    libglib2.0-0 \
    && rm -rf /var/lib/apt/lists/*

# Carpeta de trabajo
WORKDIR /app

# Copiar requirements
COPY requirements.txt .

# Instalar dependencias
RUN pip install --no-cache-dir -r requirements.txt

# Copiar el proyecto
COPY . .

# Puerto
EXPOSE 8000

# Ejecutar FastAPI
CMD ["sh", "-c", "uvicorn main:app --host 0.0.0.0 --port ${PORT:-8000}"]

```

Aquí está el Dockerfile que usamos para crear el entorno de ejecución del backend. La configuración incluye Python 3.12, las bibliotecas que necesita Tesseract OCR, los paquetes de idiomas para procesar documentos, las dependencias de Python y las instrucciones para lanzar el servidor FastAPI con Uvicorn.

7.10 Implementación de observabilidad

Para la versión final de ReceiptIA se incorporaron mecanismos de observabilidad con el objetivo de facilitar el seguimiento del funcionamiento del backend y la identificación de posibles errores durante su ejecución.

Para ello, se implementó un middleware HTTP encargado de gestionar un `request_id`, utilizado como identificador único para cada solicitud recibida. Este identificador puede ser proporcionado mediante la cabecera X-Request-ID o, si dicha cabecera no se encuentra presente, el sistema genera automáticamente un identificador mediante UUID.

```
request_id = request.headers.get("X-Request-ID") or str(uuid.uuid4())
token = _request_id_ctx.set(request_id)
started = time.perf_counter()
status_code = 500
event = "request_error"
error_type = None
```

El código presentado muestra el mecanismo utilizado para obtener o generar el identificador de cada solicitud. Además, se inicia la medición del tiempo de procesamiento, permitiendo posteriormente registrar la duración de cada operación.

Posteriormente, el middleware registra información operacional asociada a cada solicitud. Entre los datos registrados se encuentran la ruta solicitada, el método HTTP, el código de respuesta, la duración del procesamiento y el tipo de error cuando se produce una excepción.

```
try:
    response = await call_next(request)
    status_code = response.status_code
    event = "request_completed" if status_code < 400 else "request_failed"
    if status_code >= 400:
        error_type = f"HTTP_{status_code}"
        response.headers["X-Request-ID"] = request_id
        return response
except Exception as exc:
    error_type = type(exc).__name__
    logger.error(
        "Solicitud finalizada con excepción",
        extra={
            "request_id": request_id,
            "route": request.url.path,
            "method": request.method,
            "status": 500,
            "duration_ms": round((time.perf_counter() - started) * 1000, 2),
            "event": "request_error",
            "error_type": error_type,
```

La implementación permite conservar información relacionada con el `request_id`, la ruta, el método HTTP, el estado de la respuesta y la duración del procesamiento.

El identificador también se incorpora a la respuesta mediante la cabecera X-Request-ID, facilitando la correlación entre una solicitud y los registros generados durante su procesamiento.

Como complemento, ReceiptIA incorpora mecanismos de observabilidad relacionados con la evaluación de la calidad de las respuestas generadas por el componente de inteligencia artificial. Estos registros incluyen información como la puntuación obtenida, el nivel de calidad, los campos faltantes y las observaciones identificadas durante la evaluación.

```
logger.info(  
    "Calidad de respuesta IA evaluada",  
    extra={  
        "event": "ai_quality_evaluated",  
        "quality_score": puntaje,  
        "quality_level": nivel,  
        "quality_missing_fields": campos_faltantes,  
        "quality_issues": observaciones,  
    },  
)
```

El registro de estos datos permite analizar posteriormente el comportamiento del componente de inteligencia artificial, identificar respuestas que requieran revisión y detectar posibles campos faltantes o inconsistencias en los resultados obtenidos.

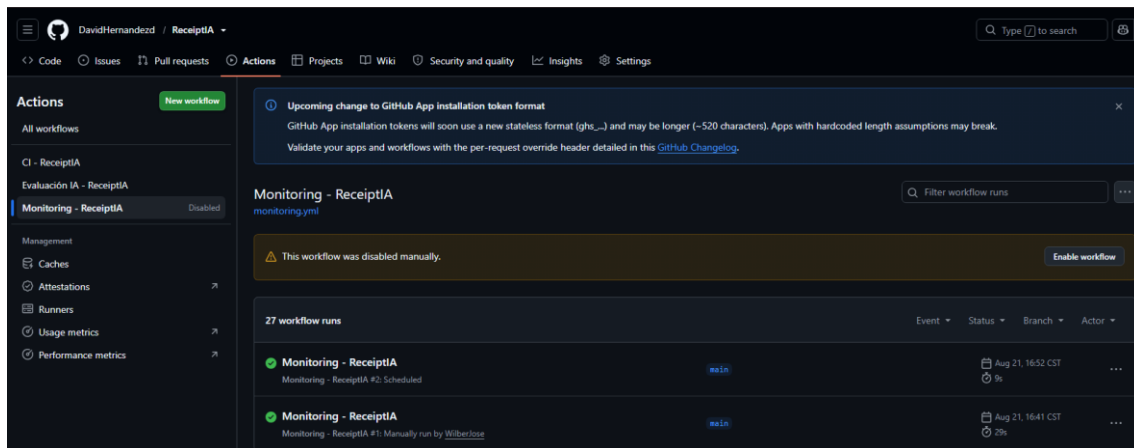
En conjunto, los mecanismos implementados permiten obtener información relevante sobre el funcionamiento del backend y facilitan el diagnóstico de problemas, evitando registrar directamente el contenido sensible de las facturas procesadas.

7.11 Implementación del monitoreo

Para la versión final de ReceiptIA, incluimos un sistema automático de monitoreo usando GitHub Actions. La idea era revisar la accesibilidad del backend y detectar posibles problemas en el servicio de forma constante.

Para hacer esto, mandamos una petición al endpoint /health, que usamos para ver cómo está el backend. Cada vez que corre, el flujo de trabajo se asegura de que el servicio responda bien y guarda los detalles importantes de esa revisión.

Aparte del código de respuesta HTTP, el sistema también nos dice cuánto tarda en llegar la respuesta y si el backend está funcionando o no.



Se ha demostrado una implementación exitosa del sistema de supervisión que se había establecido para ReceiptIA. Al llevar a cabo la verificación, el punto final /health arrojó un código HTTP 200, lo que señala que el backend se encontraba accesible. El tiempo que se tardó en recibir la respuesta fue de aproximadamente 180 ms, dato que corrobora el correcto funcionamiento del servicio durante la ejecución que se presentó.

El workflow se encuentra configurado para realizar estas comprobaciones de manera automatizada, reduciendo la necesidad de verificar manualmente el estado del backend. De esta forma, el monitoreo complementa las pruebas automatizadas y los mecanismos de observabilidad incorporados en el proyecto.

7.12 Evaluación final

Como paso final en el desarrollo de ReceiptIA, hicimos varias pruebas para ver que todo funcionara bien. La idea era también conseguir datos numéricos sobre cómo iba todo y cómo se comportaba la parte de inteligencia artificial.

Organizamos la evaluación en dos pruebas principales. Primero, hicimos un análisis comparativo de cómo funcionaba el endpoint POST /procesar, que es el que se encarga de procesar las facturas. Con esto, pudimos ver cómo respondía el sistema cuando le llegaban muchísimas peticiones a la vez, y sacamos datos sobre cuánto tardaba en responder y cuántos errores salían.

La segunda prueba se enfocó en el componente de inteligencia artificial. Usamos unos casos de prueba que preparamos para asegurarnos de que la información que sacaba de las facturas la entendía y organizaba bien.

7.12.1 Evaluación de rendimiento

Para ver qué tan bien funcionaba el sistema, hicimos 20 llamadas al endpoint POST /procesar. Mientras todo esto se ejecutaba, anotamos las respuestas que obtuvimos y varios datos sobre su rendimiento.

Entre las métricas consideradas se encuentran:

- cantidad total de solicitudes;
- solicitudes procesadas correctamente;

- solicitudes que presentaron errores;
- tasa de error;
- tiempo de respuesta P50;
- tiempo de respuesta P95;
- tiempo máximo registrado.

El empleo de estas métricas facilitó la obtención de un referente cuantitativo acerca del rendimiento del endpoint principal y la identificación de posibles áreas que podrían ser mejoradas en próximas versiones.

```
=====
RESULTADOS DEL BENCHMARK
=====
Solicitudes totales : 20
Solicitudes exitosas: 16
Errores             : 4
Tasa de error       : 20.00%
p50                 : 10682.07 ms
p95                 : 13712.31 ms
Máximo              : 17725.83 ms
=====

Archivos generados:
- benchmark_resultados.csv
- benchmark_resumen.txt
PS C:\Users\Wilber JJR\OneDrive - Universidad Gerardo Barrios\Escritorio\ReceiptIA-main>
```

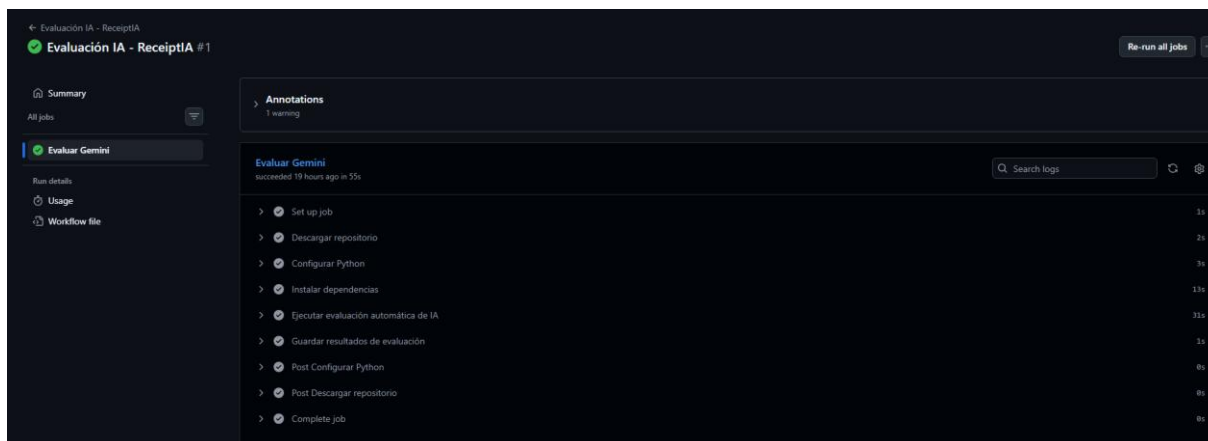
Se muestra cómo se hizo la prueba de referencia que se usó para medir qué tan bien funcionaba el punto final principal de ReceiptIA. Con esta evaluación, pudimos ver cómo respondían las diferentes peticiones y recoger datos sobre cuánto tardaban en contestar y los fallos que ocurrían al procesarlas.

7.12.2 Evaluación del componente de inteligencia artificial

Además, se hizo un análisis específico del componente de inteligencia artificial con cuatro pruebas concretas.

Estas pruebas se usaron para ver si el sistema podía identificar y ordenar bien la información importante de las facturas. Evaluamos cosas como el comercio, la fecha, el total, si estaba exento de impuestos y si había alguna irregularidad.

En cada prueba, comparamos lo que sacó el sistema con los valores que ya habíamos fijado para poder evaluarlo. De esta manera, pudimos saber cuántos casos procesó bien y sacar la precisión que se logró con todas las pruebas que hicimos.



Se presenta el procedimiento implementado para analizar el elemento de inteligencia artificial. Las pruebas realizadas facilitaron la verificación de la identificación y organización de varios datos de las facturas, así como la identificación de posibles irregularidades.

7.12.3 Análisis de la evaluación

Las pruebas que hicimos en esta etapa nos sirvieron para ver ReceiptIA desde diferentes ángulos. El análisis comparativo nos ayudó a ver cómo funcionaba el procesamiento, y la evaluación de inteligencia artificial nos permitió comprobar que el modelo hacía lo que tenía que hacer en los casos que definimos.

Guardamos los resultados para poder analizar más adelante el rendimiento, cuántos errores hubo y qué tan exacto fue el componente de inteligencia artificial. Con este estudio, podemos saber qué funciona bien y qué partes de la solución todavía hay que mejorar.

Como las evaluaciones se hicieron con los conjuntos de prueba que preparamos para el proyecto, los resultados solo muestran lo que pasó en esas pruebas. No se deben tomar como que siempre va a funcionar o ser exacto en cualquier situación que se presente.

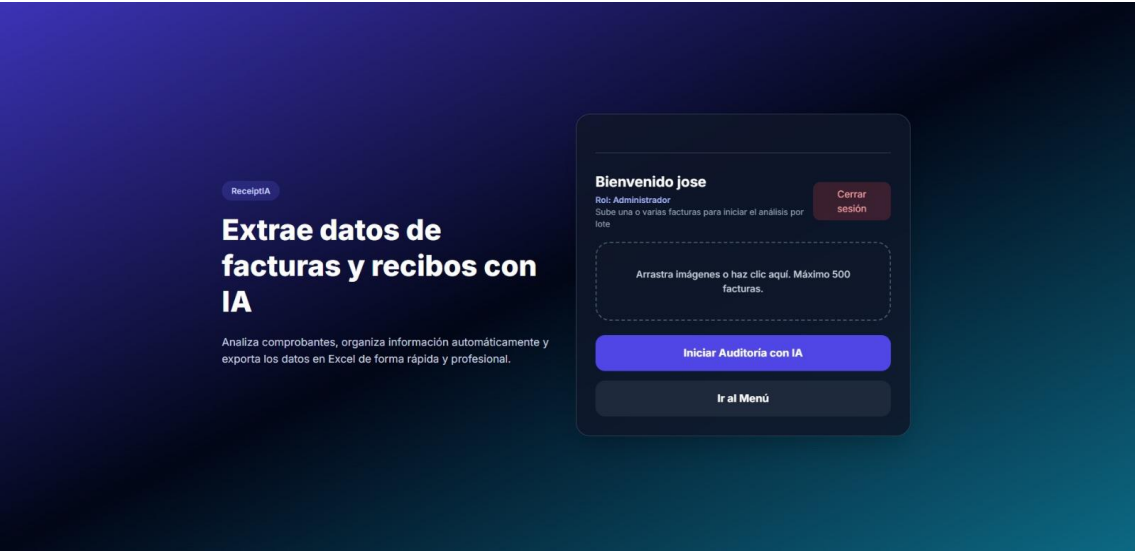
8. Resultados alcanzados

Este apartado debe centrarse en los logros alcanzados por el proyecto, y no en reiterar el proceso de desarrollo. En esta parte, emplearemos los resultados y las pruebas que hemos recopilado a lo largo de la metodología.

8.1 Funcionamiento general de ReceiptIA

Como resultado del desarrollo se obtuvo una aplicación web funcional orientada al procesamiento inteligente de facturas y comprobantes. La solución permite al usuario autenticarse, cargar documentos, procesar su información mediante OCR e inteligencia artificial y consultar posteriormente los resultados obtenidos.

La aplicación integra diferentes funcionalidades destinadas a facilitar la gestión de documentos, entre ellas el historial de facturas, organización de documentos, visualización de información procesada y exportación de datos.



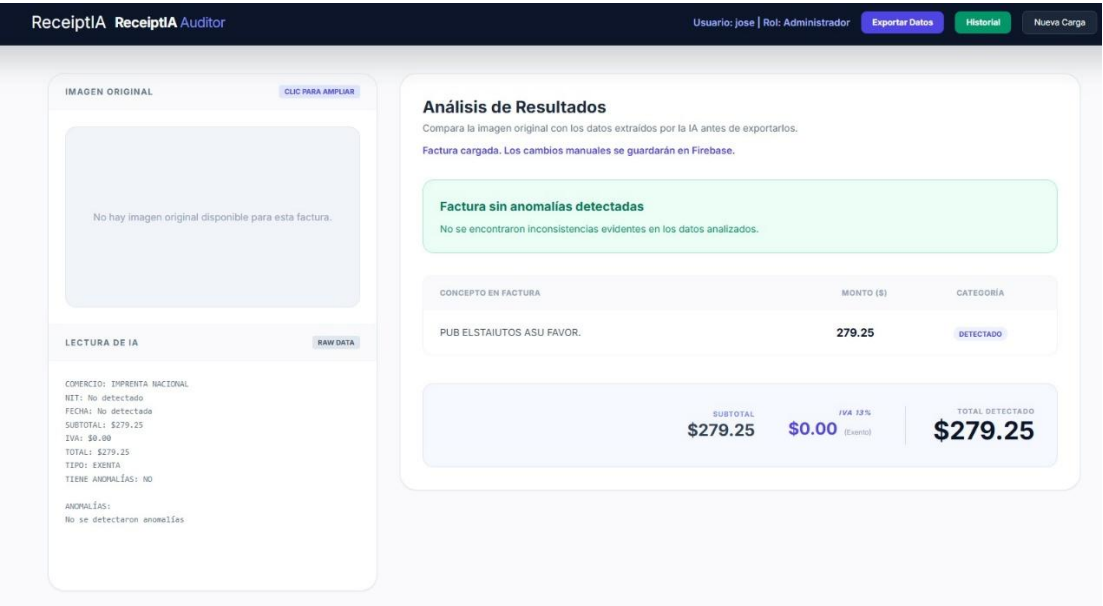
La interfaz principal de ReceiptIA y las funcionalidades disponibles para la gestión de los documentos procesados.

8.2 Resultado del procesamiento de facturas

Una de las cosas más importantes que logramos fue meter el procesamiento automático de facturas, juntando el reconocimiento óptico con inteligencia artificial.

El sistema agarra una imagen, saca el texto con Tesseract OCR y luego usa Gemini 2.5 Flash para revisar y acomodar esa información.

Gracias a esto, la aplicación puede mostrar los datos clave de la factura, como el nombre del comercio, la fecha, el NIT, el subtotal, el IVA, el total y otros datos que usamos para analizar.



El resultado que se obtiene al procesar una factura con ReceiptIA. La información contenida en el documento se extrajo y se puso en orden para que fuera más fácil de revisar y valorar.

8.3 Resultado de autenticación y almacenamiento

Firebase Authentication junto con Firestore hizo que fuera más fácil crear sistemas para manejar usuarios y guardar los datos que se generaban al trabajar con los documentos.

Así, los usuarios pueden entrar a la aplicación con sus credenciales y luego ver los documentos que están en su historial.

ReceiptIA Historial										
Usuario: jose Rol: Administrador Puede ver todas las facturas.				Volver al Dashboard	Exportar Seleccionadas	Exportar Carpeta	Exportar Todo	Limpiar Historial	Nueva Carga	
☐	21/8/2026, 21:11:40	Sin carpeta	Jimenez Ramirez	WhatsApp Image ...	IMPRENTA NACIONAL	No detectada	\$279.25	Correcta	Ver	Eliminar
☐	21/8/2026, 9:08:38 p.m.	Sin carpeta	Jonathan	modelo-de-recib...	Rojo Polo Paella tnc.	29/01/2019	\$199.65	Con anomalías	Ver	Eliminar
☐	21/8/2026, 9:05:30 p.m.	Sin carpeta	Jonathan	post_recibo.jpg	Sistema Web S.A. de C.V.	2020-03-30	\$99.50	Correcta	Ver	Eliminar
☐	21/8/2026, 20:29:38	Sin carpeta	Jimenez Ramirez	WhatsApp Image ...	IMPRENTA NACIONAL	No detectada	\$279.25	Correcta	Ver	Eliminar
☐	21/8/2026, 12:10:26 a.m.	Sin carpeta	Jonathan	WhatsApp Image ...	UNIVERSIDAD CAPITAN GENERAL GERARDO BARRIOS	2026-02-27	\$143.75	Correcta	Ver	Eliminar
☐	20/8/2026, 11:54:43 p.m.	Sin carpeta	Jonathan	factura-publicaci...	IMPRENTA NACIONAL	2010-06-21	\$279.25	Correcta	Ver	Eliminar
☐	19/8/2026, 3:00:23 p. m.	Sin carpeta	J	factura_prueba.jpg	UNIVERSIDAD CAPITAN GENERAL GERARDO BARRIOS	2026-02-27	\$143.75	Correcta	Ver	Eliminar
☐	19/8/2026, 2:54:53 p. m.	Sin carpeta	J	factura_prueba2.j...	No detectado	No detectada	\$0.00	Con anomalías	Ver	Eliminar
☐	19/8/2026, 2:44:58 p. m.	Sin carpeta	J	factura_prueba2.j...	No detectado	No detectada	\$0.00	Con anomalías	Ver	Eliminar
☐	19/8/2026, 2:44:39 p. m.	Sin carpeta	J	factura_prueba.jpg	No detectado	No detectada	\$0.00	Con anomalías	Ver	Eliminar

8.4 Resultado de las pruebas automatizadas

Implementar pruebas automatizadas nos ayudó a verificar que las partes del backend que revisamos funcionaran bien.

Corrimos las pruebas con Pytest y fueron 11 en total, todas pasaron sin problemas. Así que el porcentaje de pruebas exitosas fue del 100%.

Todo este proceso de pruebas duró casi 3.96 segundos.

Mientras corríamos las pruebas, vimos un aviso sobre una dependencia de FastAPI y Starlette que estaba un poco vieja. De todas formas, esto no causó que ninguna prueba fallara.

```
platform win32 -- Python 3.12.10, pytest-9.1.1, pluggy-1.6.0
rootdir: C:\Users\Wilber JJR\OneDrive - Universidad Gerardo Barrios\Escritorio\ReceiptIA-feature-monitoreo-evaluacion-ia
plugins: anyio-4.14.2
collected 11 items

Backend\tests\test_backend.py ..... [100%]

===== warnings summary =====
..\..\AppData\Local\Programs\Python\Python312\Lib\site-packages\fastapi\testclient.py:1
C:\Users\Wilber JJR\AppData\Local\Programs\Python\Python312\Lib\site-packages\fastapi\testclient.py:1: StarletteDeprecationWarning: Using `httpx` with `starlette.testclient` is deprecated; install `httpx2` instead.
  from starlette.testclient import TestClient as TestClient # noqa

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
===== 11 passed, 1 warning in 3.96s =====
```

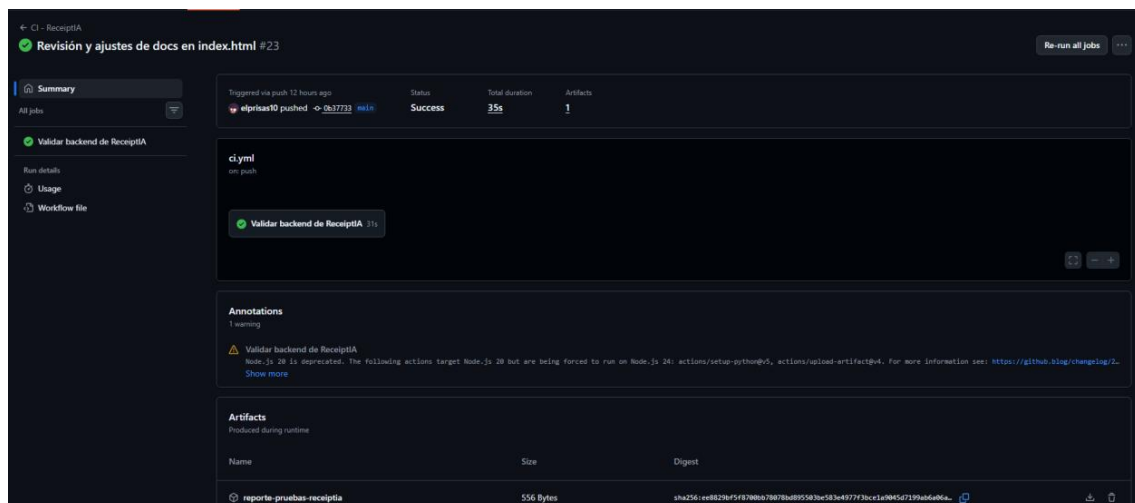
La Figura 22 muestra el resultado de la ejecución de las pruebas automatizadas. De las 11 pruebas recopiladas, las 11 fueron ejecutadas correctamente, obteniendo un 100 % de pruebas exitosas.

8.5 Resultado de la integración continua

Adoptar GitHub Actions nos ayudó a automatizar la validación del proyecto cada vez que hacíamos cambios en el repositorio.

Mirando el flujo de trabajo CI - ReceiptIA, la validación del backend se completó bien, quedó en Éxito y tardó más o menos 35 segundos.

Además, este flujo de trabajo generó el archivo reporte-pruebas-receiptia, donde se guardó la información importante de lo que pasó.



La Imagen 23 ilustra un cumplimiento exitoso del procedimiento de integración continua. El flujo de trabajo facilitó la validación del backend y produjo un informe de pruebas como producto de la ejecución.

8.6 Resultado de la evaluación del componente de inteligencia artificial

Se evaluó el componente de inteligencia artificial mediante cuatro pruebas, considerando aspectos relacionados con el comercio, la fecha, el total, el estado de exención y la identificación de anomalías.

Los cuatro escenarios definidos se procesaron correctamente en el conjunto de evaluación utilizado, alcanzando una precisión del cien por cien en los campos examinados.

```
✓ Ejecutar evaluación automática de IA
14 Ejecuta Instalar_Tesseract.bat o instala Tesseract manualmente.
15 También puedes configurar la ruta en .env con:
16 TESSERACT_CMD=C:\Program Files\Tesseract-OCR\tesseract.exe
17
18 Evaluando caso_1: Factura consumidor final correcta
19 Gemini completado en: 6740.9 ms
20 Precisión del caso: 100.0%
21
22 Evaluando caso_2: Venta exenta correcta
23 Gemini completado en: 6709.85 ms
24 Precisión del caso: 100.0%
25
26 Evaluando caso_3: Consumidor final sin IVA separado
27 Gemini completado en: 6772.61 ms
28 Precisión del caso: 100.0%
29
30 Evaluando caso_4: Crédito fiscal con IVA incorrecto
31 Gemini completado en: 9335.32 ms
32 Precisión del caso: 100.0%
33
34 =====
35 EVALUACIÓN FINALIZADA
36 =====
37 Modelo: gemini-2.5-flash
38 Prompt: 1.0
39 Casos completados: 4/4
40 Precisión general: 100.0%
41
42 Precisión por campo:
43 - comercio: 100.0%
44 - fecha: 100.0%
45 - total: 100.0%
46 - es_exenta: 100.0%
47 - tiene_anomalias: 100.0%
48
49 Reporte CSV: /home/runner/work/ReceiptIA/ReceiptIA/Backend/evaluacion/resultados/evaluacion_modelo.csv
50 Resumen JSON: /home/runner/work/ReceiptIA/ReceiptIA/Backend/evaluacion/resultados/resumen_evaluacion.json
```

Este resultado representa el comportamiento observado dentro del conjunto de pruebas utilizado. Debido al número reducido de casos, no debe interpretarse como una garantía de precisión general para cualquier factura.

8.7 Resultado del benchmark de rendimiento

Para ver qué tan bien funciona el endpoint principal de ReceiptIA, hicimos una prueba. Fueron 20 peticiones al endpoint /procesar. Con esto pudimos medir cuántas se procesaron bien, cuántos errores hubo y cuánto tardaron en responder.

En total, 16 peticiones salieron bien y 4 fallaron, así que la tasa de error es del 20%. En cuanto al tiempo, el P50 fue de 10.68 segundos, el P95 de 13.71 segundos y el tiempo más largo fue de casi 17.73 segundos.

Esto demuestra que procesar una factura lleva varios segundos y que se puede mejorar el rendimiento y la estabilidad. Parte de esto podría deberse a que dependemos de otros servicios externos para el procesamiento, sobre todo del componente de inteligencia artificial.

```
1  BENCHMARK RECEIPTIA - SEMANA 5
2  =====
3
4  Endpoint: http://127.0.0.1:8000/procesar
5  Solicitudes: 20
6  Exitosas: 16
7  Errores: 4
8  Tasa de error: 20.00%
9  p50: 10682.07 ms
10 p95: 13712.31 ms
11 Máximo: 17725.83 ms
```

8.8 Resultado de la observabilidad

Con la adopción de sistemas de observabilidad, ReceiptIA permite guardar datos operativos relacionados con las peticiones HTTP que maneja el backend. Cada petición se identifica con un `request\_id` único, lo que ayuda a vincular una solicitud con los registros que se generaron mientras se procesaba.

Estos registros incluyen información como la ruta que se solicitó, el método HTTP utilizado, el código de respuesta, cuánto tardó en procesarse, el tipo de evento y, si aplica, la clase de error que ocurrió. Además, el identificador se añade al encabezado `X-Request-ID` para poder relacionar las solicitudes con sus registros.

Esto mejora la capacidad de diagnóstico del sistema, facilitando la detección de errores y el análisis de cómo funcionan las solicitudes, sin necesidad de registrar el contenido sensible de las facturas.

```

try:
    response = await call_next(request)
    status_code = response.status_code
    event = "request_completed" if status_code < 400 else "request_failed"
    if status_code >= 400:
        error_type = f"HTTP_{status_code}"
        response.headers["X-Request-ID"] = request_id
        return response
except Exception as exc:
    error_type = type(exc).__name__
    logger.error(
        "Solicitud finalizada con excepción",
        extra={
            "request_id": request_id,
            "route": request.url.path,
            "method": request.method,
            "status": 500,
            "duration_ms": round((time.perf_counter() - started) * 1000, 2),
            "event": "request_error",
            "error_type": error_type,
        }
    )

```

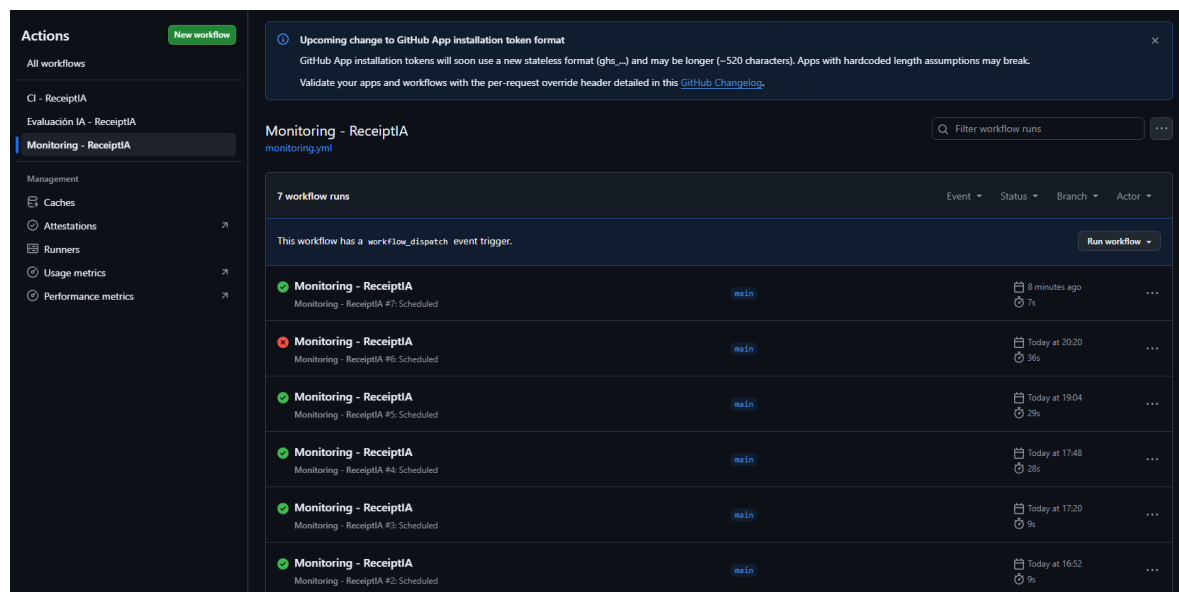
Se muestra el registro generado al procesar una solicitud HTTP. Se pueden observar elementos como el ID de la petición, la ruta, el método HTTP, el código de estado de la respuesta, el tiempo que tardó en procesarse y el tipo de evento o error. Esta información resulta útil para supervisar y examinar las acciones llevadas a cabo por el backend.

8.9 Resultado del monitoreo automatizado

Como parte de la versión final de ReceiptIA se implementó un mecanismo de monitoreo automatizado mediante GitHub Actions, cuyo objetivo es comprobar periódicamente la disponibilidad del backend.

La verificación se realiza mediante una solicitud al endpoint /health. Durante la ejecución utilizada como evidencia, el servicio respondió correctamente con un código HTTP 200 y un tiempo de respuesta aproximado de 180 milisegundos.

Estos resultados permiten comprobar que el backend se encontraba disponible durante la ejecución de la prueba y que el mecanismo implementado puede utilizarse para detectar posibles interrupciones del servicio.



The screenshot displays the GitHub Actions interface for the 'Monitoring - ReceiptIA' workflow. The left sidebar shows the 'Actions' tab with a 'New workflow' button and a list of workflows including 'CI - ReceiptIA', 'Evaluación IA - ReceiptIA', and 'Monitoring - ReceiptIA'. The main area shows the 'Monitoring - ReceiptIA' workflow with a search bar and a 'Filter workflow runs' button. Below this, a table lists 7 workflow runs. The first run is successful (green checkmark) and occurred 8 minutes ago. The second run failed (red X) at 20:20. The subsequent four runs were successful (green checkmarks) and occurred at 19:04, 17:48, 17:20, and 16:52 respectively. Each run entry includes the workflow name, the branch (main), and a 'Run workflow' button.

Workflow Name	Status	Branch	Time	Duration
Monitoring - ReceiptIA	Success	main	8 minutes ago	7s
Monitoring - ReceiptIA	Failure	main	Today at 20:20	36s
Monitoring - ReceiptIA	Success	main	Today at 19:04	29s
Monitoring - ReceiptIA	Success	main	Today at 17:48	28s
Monitoring - ReceiptIA	Success	main	Today at 17:20	9s
Monitoring - ReceiptIA	Success	main	Today at 16:52	9s

Se ha logrado un seguimiento automatizado que funciona bien. El endpoint /health devolvió un código HTTP 200 y el tiempo de respuesta estuvo alrededor de los 180 ms, lo que confirma que el backend estaba disponible cuando se hizo la prueba.

8.10 Resultado de la contenedorización

Gracias a Docker, pudimos armar un ambiente de ejecución uniforme para el backend de ReceiptIA. El contenedor incluye Python 3.12, las librerías necesarias para el proyecto y todo lo que se precisa para que funcione Tesseract OCR.

Esto hace que sea mucho más fácil ejecutar el backend en diferentes lugares y reduce las posibles diferencias entre el ambiente de desarrollo y el de producción.

```
FROM python:3.12-slim

# Evita archivos .pyc
ENV PYTHONDONTWRITEBYTECODE=1

# Muestra inmediatamente los print()
ENV PYTHONUNBUFFERED=1

# Instalar Tesseract y librerías necesarias
RUN apt-get update && apt-get install -y \
    tesseract-ocr \
    tesseract-ocr-spa \
    tesseract-ocr-eng \
    libgl1 \
    libglib2.0-0 \
    && rm -rf /var/lib/apt/lists/*

# Carpeta de trabajo
WORKDIR /app

# Copiar requirements
COPY requirements.txt .

# Instalar dependencias
RUN pip install --no-cache-dir -r requirements.txt

# Copiar el proyecto
COPY . .

# Puerto
EXPOSE 8000

# Ejecutar FastAPI
CMD ["sh", "-c", "uvicorn main:app --host 0.0.0.0 --port ${PORT:-8000}"]
```

8.11 Síntesis de resultados

Después de hacer un montón de pruebas, en ReceiptIA lograron juntar varias cosas: el manejo de documentos, la inteligencia artificial y las herramientas para desarrollar software.

Entre los principales resultados obtenidos se encuentran:

- Procesamiento automático de facturas mediante OCR e inteligencia artificial.
- 11 de 11 pruebas automatizadas exitosas.
- 100 % de precisión en los cuatro casos utilizados para la evaluación de IA.
- Implementación de autenticación y almacenamiento mediante Firebase.
- Implementación de observabilidad mediante `request_id` y registros operacionales.
- Implementación de integración continua mediante GitHub Actions.
- Monitoreo automatizado del endpoint `/health`.
- Contenedorización del backend mediante Docker.
- Benchmark de 20 solicitudes con 16 respuestas exitosas y 4 errores.
- P50 de 10.68 segundos, P95 de 13.71 segundos y máximo de 17.73 segundos en el benchmark realizado.

En resumen, lo que encontramos aquí es que la solución que se hizo logró cumplir con las funciones básicas que se habían fijado y, además, sirvió para integrar en la práctica las maneras de desarrollar, analizar y operar software dentro de un proyecto de inteligencia artificial.

9. Conclusiones generales

9.1 Conclusión general

Con ReceiptIA se pudo crear una solución tecnológica para automatizar el manejo y la organización de facturas, mezclando reconocimiento óptico de caracteres con inteligencia artificial. Esta app ayuda a que haya menos trabajo manual para obtener y ordenar la información de los documentos, lo que luego hace más fácil buscarla y analizarla.

El proyecto cumplió el objetivo de unir varias tecnologías en una sola solución. Esto incluyó un frontend web, un backend hecho con FastAPI, Tesseract OCR, Gemini 2.5 Flash, Firebase y herramientas para automatizar y desplegar todo.

9.2 Resultados concretos

Los resultados que obtuvimos muestran cómo funcionan los elementos principales que creamos. Las pruebas automáticas salieron bien, con 11 de 11 pruebas completadas con éxito. Por su parte, al evaluar el componente de inteligencia artificial, se alcanzó un 100 % de exactitud en los cuatro escenarios planteados.

Además, implementamos sistemas de seguimiento y monitoreo para poder recopilar información sobre el funcionamiento del backend y así asegurarnos de que está disponible cuando se necesita.

Al hacer la comparación de rendimiento, pudimos ver cómo responde el sistema cuando se le hacen varias peticiones seguidas. También nos dimos cuenta de que hay aspectos que se pueden mejorar, sobre todo en cuanto a los tiempos de respuesta y la cantidad de errores que se presentan.

9.3 Aportes del proyecto

ReceiptIA ofrece un aporte importante al demostrar cómo se puede usar la inteligencia artificial de forma práctica para resolver un problema relacionado con la gestión de documentos.

Esta solución viene bien para quienes necesitan organizar y consultar datos de facturas. Así se reduce el trabajo tedioso de escribir manualmente y se facilita encontrar la información que importa.

Además, el proyecto podría servir como base para hacer mejoras más adelante. Por ejemplo, se podrían añadir más tipos de documentos, usar más casos para probar el modelo, mejorar la velocidad de procesamiento y afinar los sistemas para detectar errores.

9.4 Conocimientos adquiridos

Durante el proyecto, el equipo aprendió bastante sobre inteligencia artificial aplicada, análisis de imágenes, reconocimiento óptico de texto, cómo hacer APIs REST, manejar bases de datos, autenticación y diseño web.

También se hicieron más fuertes en temas como las pruebas automáticas, la integración continua, usar contenedores, y en cómo supervisar, monitorear y evaluar el rendimiento de las aplicaciones.

Lo que vivimos nos enseñó que hacer una solución con inteligencia artificial no es solo juntar un modelo. Hay que pensar también en la validación, la seguridad, el rendimiento, el monitoreo, el mantenimiento y cómo va a funcionar todo en producción.

10. Referencias estudiadas

Gemini API. (n.d.). Google AI for Developers. <https://ai.google.dev/gemini-api/docs?hl=es-419>

FastAPI - FastAPI. (n.d.). <https://fastapi.tiangolo.com/>

Tesseract documentation. (n.d.). Tesseract OCR. <https://tesseract-ocr.github.io/>

Firebase Authentication. (n.d.). Firebase. <https://firebase.google.com/docs/auth?hl=es-419>

Firestore | Firebase. (n.d.). Firebase. <https://firebase.google.com/docs/firestore?hl=es-419>

Inc, D. (2026, August 12). *Home*. Docker Documentation. <https://docs.docker.com/>

Inc, D. (2026a, June 17). *Dockerfile reference*. Docker Documentation.

<https://docs.docker.com/reference/dockerfile/>

pytest documentation. (n.d.). <https://docs.pytest.org/en/stable/>

GitHub Actions documentation - GitHub Docs. (n.d.). GitHub Docs.

<https://docs.github.com/en/actions>

Pydantic Docs – Validation, AI agents & LogFire Observability. (n.d.). Pydantic Docs.

<https://pydantic.dev/docs/>

Charliermarsh. (n.d.). *Ruff*. Astral Docs. <https://docs.astral.sh/ruff/>